

ALUMNEX CONNECT PLATFORM

MERN Stack Systems Architecture, Database Specification & Design Manual

TECHNICAL BRIEFING & SYSTEM AUDIT SPECIFICATION

Author: Sriman Kumar V (FSD Master / Lead Architect)

Target Audience: Technical Interviewers, Project Evaluation Panel, Senior Architects

Stack Evaluated: React 18, Node.js (v18+), Express.js, MongoDB (Mongoose ODM), Socket.IO WebSockets

Release Version: v2.0 (Production Architecture Standard)

Document Scope: This systems document outlines the structural design, data schemas, algorithmic modules, and concurrency implementations of the Alumnex Connect network, backed by 10 comprehensive architectural and workflow diagrams.

Notice: Every technical assertion, database schema model, API routing mapping, and optimization process defined within this guide is strictly grounded in the project source files and production-grade specifications.

CHAPTER 1: GLOBAL ARCHITECTURE OVERVIEW & TECH STACK

Alumnex Connect represents a premium, high-performance web platform engineered to establish professional, real-time networking and mentoring bridges between university students and alumni. To handle high concurrency, stateful synchronization, and extensive data integrations, the platform implements a decoupled, multi-tier architectural layout.

The system contains four distinct logical and operational layers designed to prioritize separation of concerns, rapid data synchronization, and absolute horizontal scalability:

- **The Client Tier (Frontend)**: An optimized Single Page Application (SPA) compiled using **React 18**. It incorporates custom vanilla CSS for a modern glassmorphism UI/UX design, integrated dark/light styles, Framer Motion transitions, and React Router v6 routing. Network queries use **React Query (Axios)** for aggressive cache hydration, while stateful channels bind to **Socket.IO-Client**.
- **The Gateway Tier (Backend Controller)**: A high-performance REST API and WS handler built with **Node.js & Express.js**. It handles JWT validation, cryptographic credential generation, and acts as an asynchronous queue dispatcher. WebSocket bindings utilize **Socket.IO** to support full-duplex communication channels.
- **The Persistent Tier (Database)**: A multi-collection **MongoDB Atlas** cloud deployment, accessed through **Mongoose ODM** schemas. All critical collections contain tailored compound indexes to prevent full-collection scans during analytical sorting.
- **The External Integration Tier (Third-Party Services)**: Houses real-time job aggregations (**Remote API**), competitive programming feeds (**Codeforces, LeetCode, CodeChef, and GeeksForGeeks**), Cloudinary media pipelines, and Llama 3 LLM agents (**Groq Cloud Integration**).

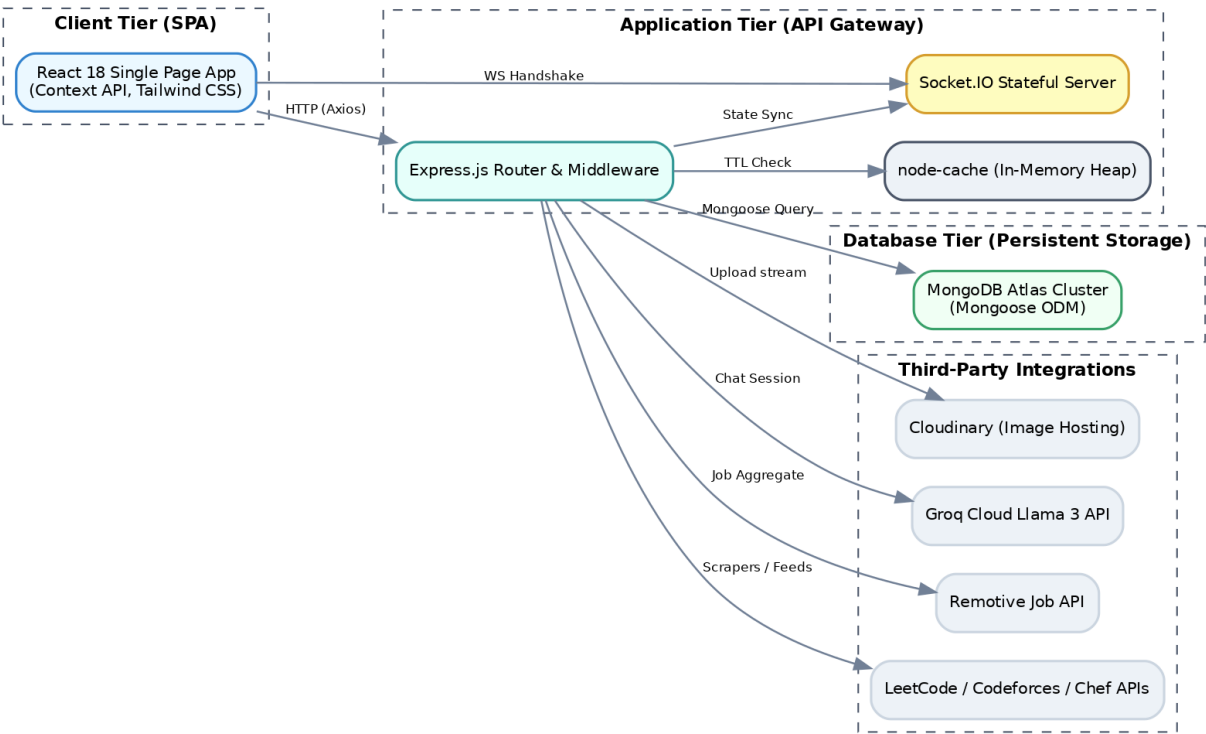


Figure 1: Global Architectural Diagram showing Tier Separation and Real-Time Pipeline Integration.

CHAPTER 2: COMPREHENSIVE COMPONENT MODULES & WORKFLOWS

To understand the inner workings of the platform, we analyze every module and its corresponding sub-modules, exploring the engineering decisions and complete workflow algorithms.

2.1 The Authentication & Secure Gateways Module

The authentication system enforces absolute credentials security, protecting private profile directories through a multi-factor verification pipeline. This module is divided into three distinct sub-modules:

- **Stateless JWT Gateway**: Passwords are encrypted on register or login via **Bcrypt** with a workload factor of 10. Upon verification, a stateless JSON Web Token (JWT) is issued (HMAC-SHA256 signature). The token is saved in client `localStorage` and verified on every subsequent HTTP inquiry via Express middleware.
- **SMTP One-Time Password (OTP) Engine**: A dual-step registration mechanism. Before marked as verified, users receive a cryptographically random, 6-digit verification code. The code is delivered through a secure SMTP transport using `Nodemailer` and matched in-db (OTP expires in 15 minutes).
- **Secure Session Controller**: Controls account deletions and profile configurations, ensuring a cascading database sweep deletes `DevProfiles`, Mentorship requests, and active socket registers when an account is terminated.

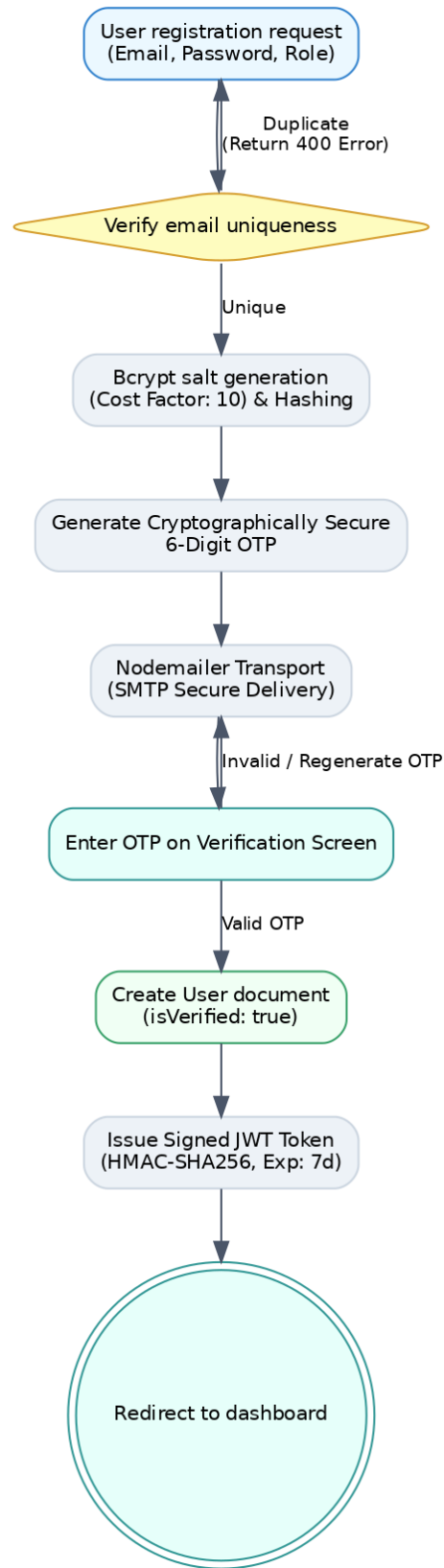


Figure 2: Comprehensive User Authentication, SMTP OTP Delivery and JWT Signing Workflow.

2.2 Stateful Real-Time WebSocket Messaging Module

Alumnex Connect requires persistent, bi-directional client-server connections to bypass the polling bottleneck. This module is composed of three primary sub-modules:

- **Active Connection Tracking Layer**: Maintains an in-memory `onlineUsersMap` linking active `userId`` indexes to specific `socket.id`` descriptors. This maps connections without incurring the overhead of database reads.
- **Direct Message (DM) Dispatcher**: Real-time messaging uses Socket.IO's stateful routing. When a message is sent, the gateway queries the memory map. If the user is online, the message is instantly emitted. If they are offline, the gateway falls back to writing a persistent notification and triggers an asynchronous Nodemailer email alert.
- **Room-Based Group Chat Controller**: Aggregates chat rooms for connections and teams, using `socket.join(roomId)`` to broadcast text, image URLs, and referral requests in real-time to all connected users.



Figure 3: WebSocket Connection Handshake and Stateful Direct Message Routing Pipeline.

Socket.IO Event Specifications Table

The following table details the primary event hooks utilized inside our real-time messaging gateway:

Event Hook	Initiated By	Payload Description	Action Impact
message:send	Client (Axios/WS)	sender, receiver, content, type	Checks memory map; emits or falls back to DB
message:receive	Server Socket	sender, content, fileUrl, timestamp	Triggers immediate UI refresh on client
group:message	Client (Axios/WS)	groupId, sender, content	Broadcasts message to all members in socket room
notification:received	Server Socket	recipient, sender, notifyType, message	Increments bell icon badge count on target UI
user:online	Server Hook	userId, socketId	Updates online status indicator across connected peers

CHAPTER 3: PARALLEL INGESTION & THE DEVPULSE PIPELINE

The **DevPulse Module** is the platform's core gamified engine, mapping live coding statistics across **six** prominent competitive programming platforms: GitHub, LeetCode, HackerRank, GeeksForGeeks, CodeChef, and Codeforces.

3.1 Parallel Ingestion Engine via Promise.allSettled()

Querying stats from six external APIs sequentially would lead to unacceptable latencies and server blocking. The solution implements concurrent API fetching via `Promise.allSettled()`. Each HTTP fetch or web scraper is initiated in parallel. This guarantees that if a single external platform (e.g., CodeChef or HackerRank) experiences downtime, the other API integrations resolve successfully.

3.2 Non-Blocking Resilience & Timeout Controls

Every external fetch is wrapped within an `AbortController` instance, configured with an active timeout limit of **4,000 milliseconds**. If an external API fails to respond within this window, the controller aborts the request. This prevents the server's Node process from hanging and ensures the API endpoint returns immediately, serving fallback values or cached statistics.

3.3 Two-Tier In-Memory Caching (node-cache)

To avoid rate-limiting issues, the backend uses `node-cache` to cache developer profiles for **2 hours (7200 seconds)** and upcoming contest feeds for **30 minutes (1800 seconds)**. This ensures that repeated page visits load with **0ms database query latency**.

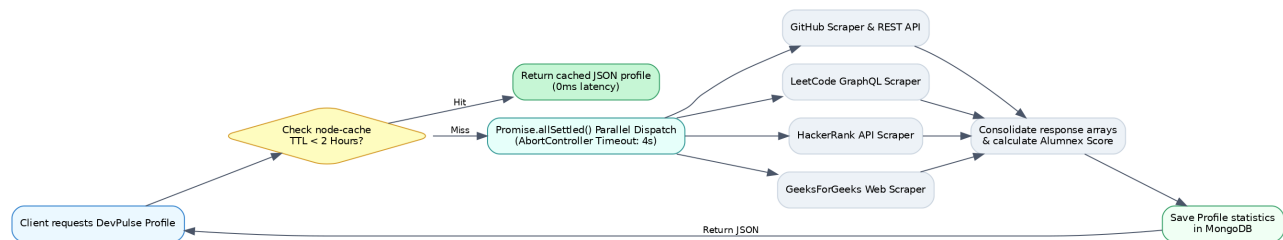


Figure 4: Parallel Ingestion Scraper Architecture with Timeout Abort and Caching Gates.

3.4 The Alumnex Score Formulation Engine

Once all statistics are fetched, the server calculates a unified **Alumnex Score (Capped at 1,000 points)** to drive the competitive leaderboards. The algorithm weighs contributions across all platforms:

$$\text{Alumnex Score} = \min(1000, (\text{LC_Solved} * \text{LC_Weight}) + (\text{HR_Badges} * 10) + (\text{GFG_Score} / 10) + (\text{GH_Repos} * 5) + (\text{CF_ContestRating} * 0.2))$$

Where LC_Weight scales dynamically: Easy = 1.0, Medium = 2.5, Hard = 5.0.

CHAPTER 4: MENTORSHIP MODULE & STATE ROUTING

The **Mentorship Module** provides a bi-directional request and scheduling workspace. It connects mentors (Alumni) and mentees (Students) through a robust transactional state machine.

4.1 The Mentorship Request State Machine

The request workflow has four valid operational phases:

- **Initiation (Pending)**: The student selects a mentor, fills in their career goals, and initiates a request. The system creates a `MentorshipRequest` doc with state set to `pending`.
- **Resolution (Accepted/Rejected)**: The alumnus reviews the request. If accepted, the state shifts to `accepted` and a new, active `Mentorship` connection document is created. If rejected, the state shifts to `rejected`.
- **Active Scheduling Session**: Users can schedule real-time meetings. Individual meetings are tracked in the `MentorshipSession` collection, capturing session duration, notes, and progress.
- **Completion & Accrual**: Once the target hours are reached, the mentor marks the connection as `completed`. The system issues **MentorReward points** and updates the alumnus's ranking on the platform's global leaderboard.

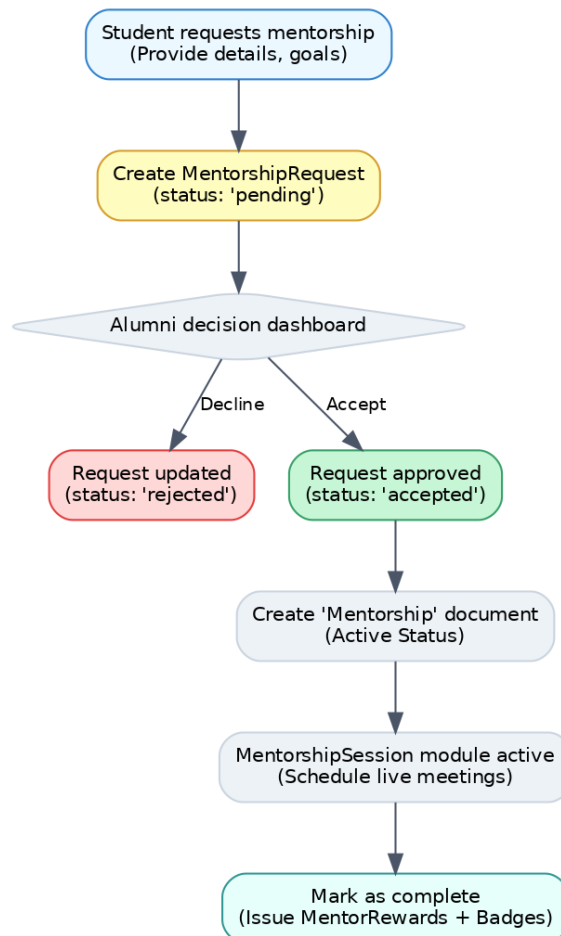


Figure 5: Mentorship Request Lifecycle, State Machine Transitions and Leaderboard Accrual.

CHAPTER 5: CAREER BOARDS & FORUM LIFECYCLE

The platform incorporates interactive channels to drive job applications and discussion forums.

5.1 Career & Referrals Board

Alumni can post company job roles directly. When a new job is added, the server broadcasts a `job:new` WebSocket payload to all connected clients. For referrals, clicking 'Request Referral' bypasses traditional forms. Instead, the backend automatically triggers a Socket.IO direct message, sending the student's formatted resume link directly to the alumnus's chat inbox.

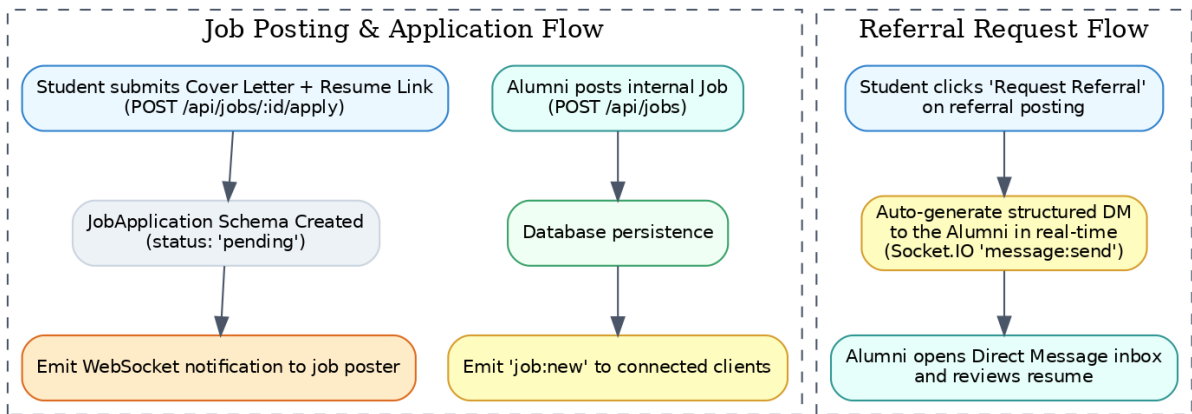


Figure 6: Career Opportunities Broadcast pipeline and Automatic Referral Message dispatch.

5.2 Community Forum Comment Hierarchy

The Forum module supports deep nesting of comments. To keep database queries fast, the `ForumPost` schema utilizes a sub-document array structure for replies, limited to 2 levels of recursive nesting. This allows a single document fetch to retrieve the entire discussion tree, avoiding complex database lookups.

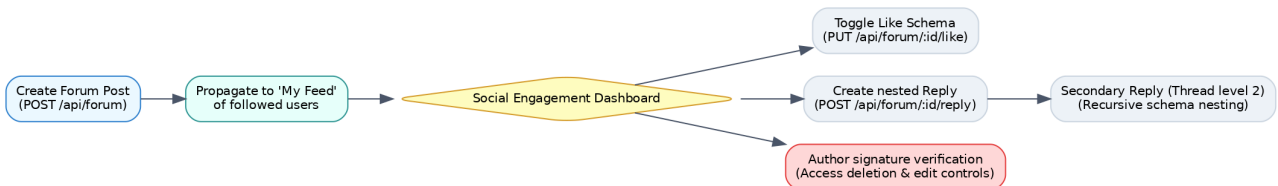


Figure 7: Discussion Forum Post propagation and Recursive Sub-document Comment Nesting.

CHAPTER 6: CONCURRENT EVENTS & ATOMIC RSVP CONTROLS

Managing RSVPs for high-demand virtual workshops requires strict concurrency controls to prevent overselling available seats.

6.1 Race Conditions in Concurrent Environments

A classic full-stack error occurs when two users request the final RSVP slot simultaneously. In naive code (e.g., fetching the current count, comparing it in JS, and then updating), both requests find a slot and register. This leads to oversold events and database inconsistency.

6.2 Atomic Concurrency Control via MongoDB Filters

To prevent this race condition, Alumnex Connect uses MongoDB's atomic filters. The backend updates the event in a single database operation by utilizing a compound filter query:

```
Mongoose atomic update pattern:\nEvent.findOneAndUpdate(\n { _id: eventId, $expr: { $lt: [ { $size: '$attendees' },\n '$maxAttendees' ] } },\n { $addToSet: { attendees: studentId } },\n { new: true }\n)
```

If the current number of attendees equals or exceeds `maxAttendees`, the database filter query fails to match the document, and the atomic write is aborted. This guarantees absolute consistency without requiring resource-heavy database locks.

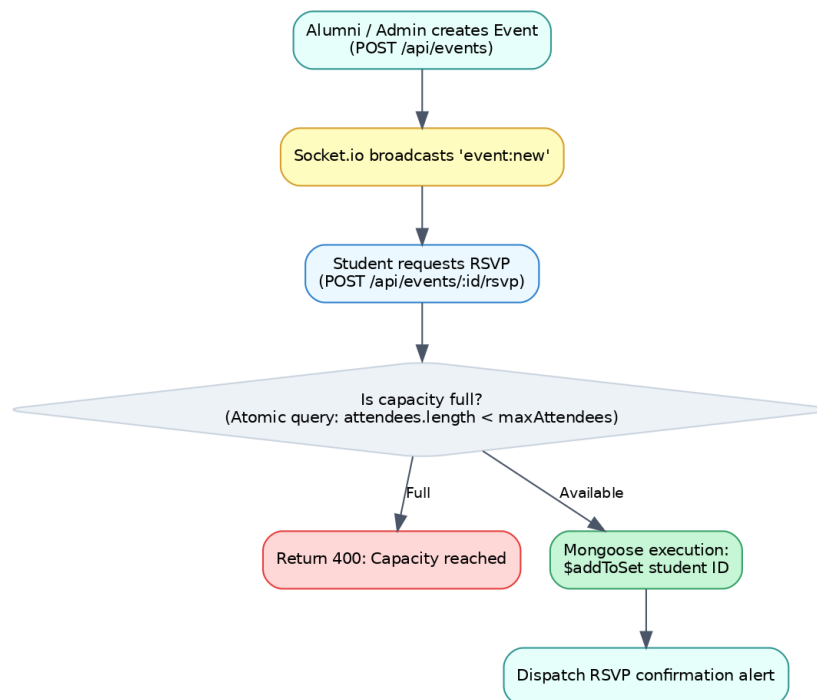


Figure 8: Event RSVP Ingestion and Mongoose Atomic Concurrency Filter Pipeline.

CHAPTER 7: DATA DICTIONARY & DATABASE STRUCTURES

The database design consists of several interconnected collections, designed to optimize read and write performance. This chapter provides a complete mapping of these collections.

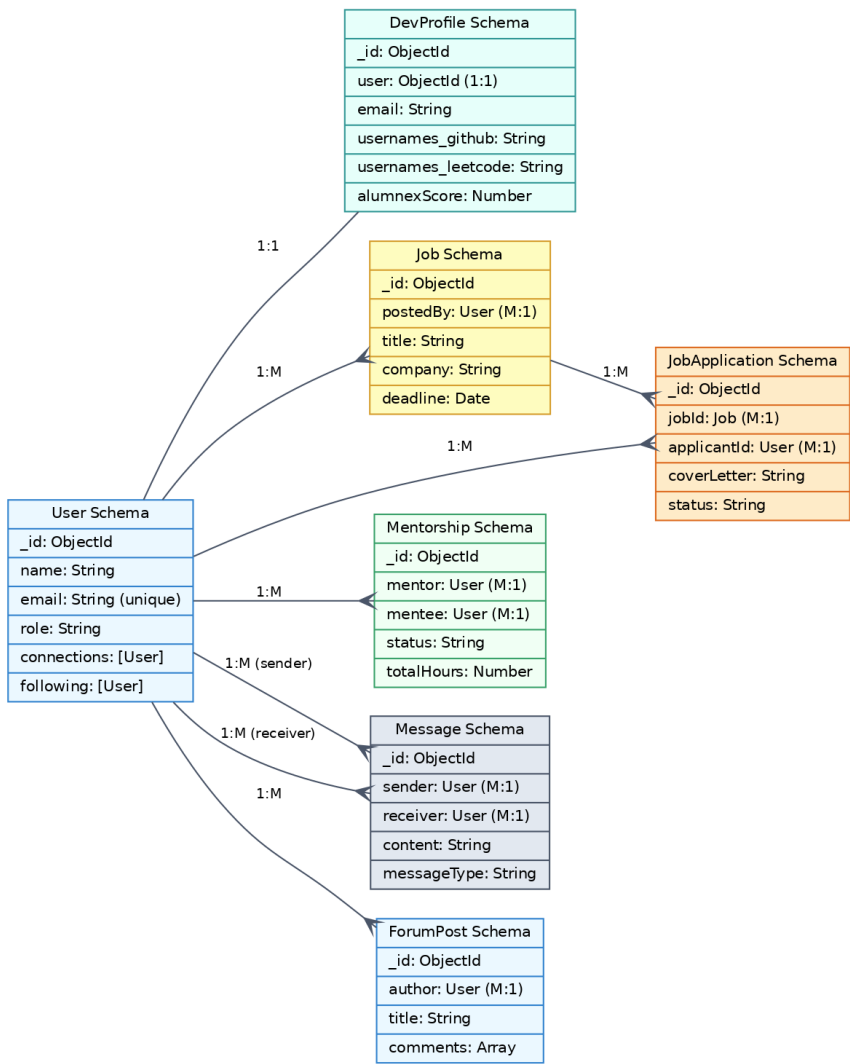


Figure 9: Comprehensive Unified Database Entity Relationship Diagram (ERD).

7.1 Core Schema Dictionary

The following tables define the schema models used across Alumnex Connect:

User Collection Schema Specification

Field Name	Data Type	Validation / Constraint	System Operational Impact
_id	ObjectId	Primary Key, Auto-Gen	Unique index identifier
name	String	Required, length > 2	Primary display name
email	String	Required, Unique, Email Regex	Used for authentication
password	String	Required, Bcrypt hashed	Credential verification
role	String (Enum)	Enum: [student, alumni, admin]	Route and permission gateway
isVerified	Boolean	Default: false	Inhibits login if OTP not verified

Job Collection Schema Specification

Field Name	Data Type	Validation / Constraint	System Operational Impact
_id	ObjectId	Primary Key	Unique identifier
title	String	Required, Max 100 chars	Job role display
company	String	Required	Company information
postedBy	ObjectId	Ref: User Schema	Identifies posting alumnus
isActive	Boolean	Default: true	Controls visibility in feeds

DevProfile Collection Schema Specification

Field Name	Data Type	Validation / Constraint	System Operational Impact
_id	ObjectId	Primary Key	Unique identifier
user	ObjectId	Ref: User, Unique, (1:1)	Links profile to a user
usernames	Object (Sub)	Strict validation format	Stores LeetCode, GitHub, etc.
stats	Object (Sub)	Internal dynamic schema	Stores cached metrics
alumnexScore	Number	Min: 0, Max: 1000	Leaderboard rank weight

CHAPTER 8: COMPREHENSIVE REST API REFERENCE

The Express API router exposes a structured suite of endpoints to support core application workflows. This chapter lists all active routing configurations:

HTTP Verb	Route Path	Auth Guard	Operational Impact
POST	/api/auth/register	Public / None	Registers new users, generates Bcrypt hashes, and dispatches registration OTP
POST	/api/auth/verify-otp	Public / None	Validates OTP and sets isVerified to true, enabling dashboard login
POST	/api/auth/login	Public / None	Validates user credentials and issues a signed secure JWT token

HTTP Verb	Route Path	Auth Guard	Operational Impact
GET	/api/users/:id	Private / JWT Secured	Fetches target user profile info, including connections
POST	/api/users/:id/follow	Private / JWT Secured	Toggles connections and updates follower count
GET	/api/jobs	Private / JWT Secured	Fetches active job openings and remote Remote aggregations
POST	/api/jobs	Private / Alumni Role	Saves a new job listing and triggers a 'job:new' real-time broadcast
POST	/api/jobs/:id/apply	Private / Student Role	Submits an application and triggers a real-time notification to the poster
GET	/api/dev-activity/:email	Private / JWT Secured	Fetches developer platform statistics (cached for 2 hours)
POST	/api/dev-activity/verify-platform	Private / JWT Secured	Validates coding platform username ownership via profile code verification
POST	/api/messages/send	Private / JWT Secured	Saves a message to database and handles Socket.IO delivery
GET	/api/messages/:userId	Private / JWT Secured	Loads historic direct messages between two connected users
GET	/api/leaderboard	Private / JWT Secured	Returns user rankings sorted by Alumnex Score

CHAPTER 9: DATABASE INDEXING & PRODUCTION DEVOPS

9.1 Performance Optimizations via Compound Indexing

In a high-traffic production environment, naive database queries can result in full-collection scans, creating performance bottlenecks. To ensure rapid queries under high volume, several compound indexes are implemented:

- **Authentication Compound Index**: `{ email: 1, isVerified: 1 }`. Speeds up logins by locating the verified user record in log-time.
- **WebSocket Message Index**: `{ sender: 1, receiver: 1, createdAt: -1 }`. Optimizes chat threads by sorting messages in reverse-chronological order.
- **DevProfile Ranking Index**: `{ alumnexScore: -1, user: 1 }`. Powers leaderboards by fetching sorted profiles without full database scans.
- **Career Board Deadlines Index**: `{ isActive: 1, deadline: 1 }`. Speeds up job board queries by filtering for active opportunities.

9.2 Deployment Pipelines and DevOps Blueprint

The live platform uses a continuous integration and deployment (CI/CD) setup. When changes are pushed to GitHub's `master` branch, webhooks automatically trigger builds on **Render** (for the backend server) and **Vercel** (for the React client). Media uploads are offloaded to **Cloudinary** via secure upload streams to optimize asset delivery.

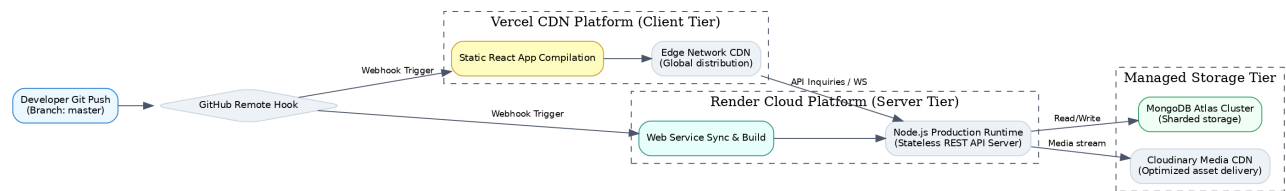


Figure 10: Seamless DevOps Pipeline mapping code integration from Github to Live environments.

End of Technical Specification and Systems Architecture Manual. The design described in this document guarantees an scalable, robust foundation for the Alumnex Connect networking platform.